IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 21 October 2025, accepted 19 November 2025, date of publication 26 November 2025, date of current version 5 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3637221

RESEARCH ARTICLE

AI-Powered Unit Test Generation via Multi-LLM Chaining: A Case Study With GPT-4o, Gemini, and Claude-3.5

CHANDAN KUMAR[1,2], USHA SRI PONAKA[1], PULA V. LAKSHMI NARASIMHA NAIDU[1], PYATLO BHUVANESWARI[1], MUKESH PRASAD[3], AND SUNIL KUMAR SINGH[4], (Member, IEEE)

[1]Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amaravati Campus, Amaravati 522503, India
[2]Amity Institute of Information Technology, Amity University Jharkhand, Ranchi 834004, India
[3]School of Computer Science, University of Technology Sydney, Sydney, Ultimo, NSW 2007, Australia
[4]Department of Computer Science and Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal 576104, India

Corresponding authors: Sunil Kumar Singh (sunil.singh@manipal.edu) and Chandan Kumar (chandan.jha150286@gmail.com)

ABSTRACT Software testing is a crucial activity in the software development cycle, as it verifies code correctness, reliability, and maintainabilily. Unit testing involves verifying the correctness of the individual components of a program. Manually writing these tests is resource-intensive and time-consuming. Researchers have proposed various automated test generation methods to overcome this problem. Large Language Models have shown great success in automatically generating unit tests recently. Although a single LLM configuration can produce satisfactory responses, there are possible risks to the effectiveness of the generated tests. We present a novel method, LLM Chaining, that uses the collaboration of several LLMs, namely Gemini, GPT-4o, and Claude-3.5 Sonnet, to collaborate and iteratively enhance tests. We began by testing a single LLM writing unit tests, but after noting their suboptimal performance, we explored LLM chaining as a method to enhance the accuracy and coverage of the produced tests. Gemini and GPT-4o yielded the most reliable results of all the configurations we tested. Under this configuration, the LLM system uses Gemini to generate the initial JUnit tests, then sends the generated results to GPT-4o to improve accuracy. The HumanEval dataset was used to create JUnit test cases for this study. JaCoCo is used to collect coverage statistics, and PIT is used for mutation-based testing to measure an LLM's ability to detect faults in tests. Our approach achieved 99.05% branch coverage, 90.48% line coverage, and 94.32% mutation coverage. In contrast, Randoop, a well-known automated test generation tool, obtained 68.64% branch coverage and 78.84% line coverage. This demonstrates how multi-step LLM refinement works well to advance automated test generation.

INDEX TERMS Software testing, unit testing, LLM, Gemini, GPT-4o, Claude-3.5 Sonnet, LLM chaining, JUnit, JaCoCo.

I. INTRODUCTION

Software engineering is a specified and measurable process for designing, implementing, and maintaining software [1]. It serves as a process to ensure that the software is reliable, works as intended, and meets user desires. In software engineering, one activity is more important than others—software testing, which is the process by which we check that software is built correctly, operates as expected, and behaves as rationally intended [2]. Software testing is a formal assessment of a system, or an element of a system, to identify defects, find nonconformity with requirements, and assert anticipated behavior. Testing is a vital phase of the software development life cycle since it contributes to minimizing defects in production software by verifying the software prior to deployment into production [3].

Unit testing is the lowest level of software testing. It aims to ensure that individual functions or units are correct.

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Developers normally write test cases automatically before writing the functionality to ensure that each unit is as required. In a standard unit testing cycle, tests are run; if they succeed, then the functionality either is right or already exists. If a test fails, the developer makes changes to the code and repeats the tests until they succeed. After passing the tests, project stakeholders are able to inspect the results in terms of feedback. This is a process that is especially valuable and practical for medium- to large-sized projects [4]. Unit tests minimize the risk of releasing faulty software, make debugging easier, and allow early bug detection [5]. They also improve the software and reduce maintenance cost in the long run.

Manual unit testing is time-consuming and involves a high degree of error. Test cases are produced by developers, tests are executed, and output is inspected through the console—which is not very scalable for systems that are not trivial. In addition, the time it consumes to test manually is more and significantly raises software construction cost [6].Researchers have investigated automation as a means of enhancing test coverage and efficiency.

Standard automated testing techniques, such as symbolic execution, evolutionary algorithms, and model checking, have been developed through extensive path exploration within a program. However, such conventional techniques might not reflect the underlying intent of the code and hence restrict their applicability in comparison to human-generated tests [7]. They continue to struggle with generating high-quality test cases that accurately capture real-world applications even though they assist in widening coverage.

Automated test generation is addressing some of the problems associated with manual testing by saving time and improving scalability. Secondly, sites like GitHub and Stack Overflow have transformed testing in the way we are now able to share our tools, our expertise, and our methods to the entire globe [8]. These sites facilitate quicker iteration and more automation in test generation, leading to reduced development time and improved software quality.

Researchers are beginning to use machine learning (ML) and deep learning (DL) technologies to generate tests as a reaction to the issues caused by modern-day software complexity. These technologies use learning from code to develop better tests. As DL-based technologies improve coverage, these approaches do have some limitations: they struggle with scalability and they don't have enough training data. These limitations prevent them from understanding code intent fully and designing the best test cases possible [9].

The emergence of Large Language Models (LLMs)—such as OpenAI's GPT-3 and Codex—has been a major boost to software testing. LLMs are extremely good at understanding codes and doing tasks in natural language, making them perfect candidates for automatically writing unit tests. Additionally, LLMs can rewrite tests when given feedback about errors, making the tests more trustworthy. This technology is encouraged by open platforms such as GitHub that support creativity and encourage community gains and improvements [8].

Regardless of this development, the LLM-based approaches still present substantial challenges. Single-pass generation with static prompts often creates test cases that either will not compile, have no meaningful coverage or provide poor practices [10]. Although quick engineering is widely used, manual testing cannot be completely replaced by it. Issues with privacy, explainability, and security continue to exist [12]. Current approaches are deficient in providing the iterative improvement and collaborative thinking necessary to produce complete and trustworthy tests.

We introduce an approach designed to fill the gaps in automated unit test generation under the name of LLM Chaining. LLM Chaining allows multiple large language models (LLMs) to collaborate in a structured and iterative way so that test cases can be generated in high quality, and now the high quality is not based just on single-step generation, but each of the strengths of the different models like Gemini, GPT-4o, Claude-3.5 Sonnet, etc.

The key contributions of this research are:

- We perform a comprehensive coverage analysis using both single LLM and LLM chaining approaches, evaluating their effectiveness through line and branch coverage metrics.
- We perform mutation testing to assess the fault-detection capability and robustness of LLM-generated test cases compared to refined LLM chaining methods.
- We compare LLM-generated test cases with the non-LLM (Randoop) baseline to demonstrate the relative performance of AI-driven versus conventional automated test generation methods in terms of coverage.

We evaluated this technique using the HumanEval dataset and measured coverage metrics with JaCoCo. Our automated pipeline ran 3,508 test cases, recording 99.05% branch coverage, 90.48% line coverage, and 94.32% mutation coverage, including regions of previously unexplored code. These findings illustrate that LLM Chaining has the potential to greatly enhance automated unit test generation, resulting in stronger and more maintainable software.

II. BACKGROUND AND RELATED WORK

A. THE SIGNIFICANCE OF SOFTWARE TESTING

In the software development life cycle, software testing is a crucial step that guarantees systems meet user needs and adhere to industry standards. Testing is common to measure system functionality, reliability, security, and performance under various scenarios as systems grow larger and more complex, particularly with features like microservices. Early detection of defects reduces repair costs and prevents system failures [13]. More recently, advancements in high-quality Large Language Models (LLMs) have addressed the present constraints of assessing intricate, real-world applications, opening up exciting opportunities for test generation automation [14].

While performance testing ensures that a software program remains responsive when user demand increases, security testing protects sensitive data from security incidents. Regression testing confirms that updates made during a continuous integration and delivery (CI/CD) process do not adversely affect software. When combined, these testing methods lower maintenance costs, safeguard usability, and provide assurance that software systems continue to be reliable and flexible in a digital world [15].

On a larger scale, automating unit testing is still difficult, despite its importance. With all of the microservices and frequent CI/CD workflows, manual testing may become unmanageable. Writing and maintaining tests by hand for thousands of services is not feasible. Delays and inadequate testing result from this. Advances in AI and LLMs, like Claude 3.5, have produced more efficient automated unit tests, which have improved test success and increased coverage [16]. However, current AI techniques frequently result in tests that are either inadequate or poorly contextualized, highlighting the necessity for sophisticated strategies to handle system complexity.

B. UNIT TEST GENERATION

Automated unit testing is turning out to be a cost-effective way to improve software quality and decrease manual testing burden. The landmark in this regard is the research paper of Fraser and Arcuri [17] entitled "Whole Test Suite Generation." They used the EvoSuite tool for introducing search-based software testing (SBST) for Java. EvoSuite significantly boosted test coverage and fault detection, paving the way for tools like EvoMaster [17]. The integration of SBST (Search-Based Software Testing) in Web API testing has paved the way for new research directions. Despite the development of more advanced methods, SBST remains effective in generating well-structured and practical test cases. Modern tools apply various techniques, such as random data generation, feedback-directed testing, and combinatorial testing, to create JUnit tests for software developed in different programming paradigms.

Advanced techniques like symbolic execution and concolic testing have also increased the space of test generation that is possible for applications of C and Java. Constraint-based reasoning has also had its share of influence, particularly in the generation of tests for programs from Java bytecode. These methods, however, are confronted with challenges in representing complex constraints, interaction with SQL, and support for dynamic programming languages. Also, they are generally rigid to fit runtime environments or specific research inputs. To address these shortcomings, dynamic and adaptable frameworks have been established. These types of frameworks generate low-error, coherent test cases by addressing Java bytecode and opcode directly, leading to more reliable unit testing approaches [18].

Despite these advancements, scalability is a significant concern. Symbolic execution may be plagued by path explosions in complex systems, yet genetic algorithms, while beneficial, are computationally costly, as noted by Fraser and Arcuri [19]. Other methods employ UML sequence diagrams to specify component interaction in order to obtain unit tests that verify functional and non-functional requirements, e.g., error handling in dynamic systems [20].Systematic reviews indicate that quality is guaranteed with assurance of reduced manual effort with automated tools. Automated tooling, however, suffers from the absence of common evaluation criteria, rendering findings generalizable across tools problematic. Variation in tool capability for dealing with functionality, scalability, and integrability further complicates the picture.

Recent surveys mitigate these issues by focusing on tools that have feedback mechanisms and systematic test generation techniques, and they provide a direction towards future research [21]. Unit testing has also become extremely important in fields like reinforcement learning, where the reliability of algorithms is of the highest priority. For instance, one of the most popular Python packages for reinforcement learning had 95% code coverage through automated tests, which were themselves tested against reference implementations. Reproducible research was also stressed, enabled by stable interfaces and quality documentation [22]. Although these methods work well in most situations, they typically depend on pre-curated datasets, limiting their applicability to other codebases. Advances such as iterative LLM-based test generation are more flexible and scalable, thus overcoming these previous limitations.

C. LARGE LANGUAGE MODELS FOR UNIT TEST GENERATION

Large Language Models (LLMs) have introduced a new era for automated coding tasks, such as unit test generation, by being able to produce syntactically correct code and test cases. Chen et al. [23] provided early evidence of what LLMs could do when they produced JUnit-like test cases from natural language specifications or code snippets with Codex. The test cases provided by Codex were syntactically correct tests in [23]. Later work by Liu et al. [24] leveraged GPT-3.5 and was able to generate well-formed unit tests, especially in simpler contexts, but ran into test completeness and coverage issues.

More recent breakthroughs with models like GPT-4 have enhanced executable test generation [24]. Nonetheless, even GPT-4 suffers from issues caused by single-pass inference and limits to prompting styles, resulting in bugs such as incomplete edge case coverage, redundant assertions, and common anti-patterns in test structure [24]. These constraints, such as not being able to iteratively improve tests or take advantage of collaborative reasoning between models, encourage further sophisticated methods. Compiling a direct comparison of the efficacy of different LLMs, Patel et al. [25] discovered that Gemini is better suited to writing code, whereas Claude-3.5 Sonnet is better at detecting errors in code-generation attempts [24]. Together, these works

underscore the ability of LLMs to lower manual testing workloads, albeit they still generate tests with prevalent anti-patterns (e.g., redundant assertions, absent edge case assertions) [24].

To classify test generation using LLMs, we present the following taxonomy:

- **One-shot LLMs**: Produce tests in a single inference step with static prompts, as in early applications of Codex [23]. These are efficient but tend to be incomplete and context-insensitive.
- **Feedback-tuned LLMs**: Use feedback loops to iterate towards test case improvement, as investigated with GPT-3.5. These better maintain quality but are still bound by single-model limitations.
- **LLM Chaining**: A multi-model, iterative approach in which several LLMs (e.g., Gemini and GPT-4o) jointly construct and iteratively improve tests, increasing correctness and coverage.

The progress of testing with large language models (LLMs) indicates that an "LLM Chaining" approach may form an iterative procedure using several LLMs. In this method, later models (like Gemini and GPT-4o) start from the output of earlier models in the pipeline and hence potentially improve on correctness and test coverage. This iterative and multi-model process may break away from conventional single-model and single-inference techniques and provide a new model for creating tests innovatively and iteratively in different software settings. By addressing the long-standing limitations of even sophisticated models such as GPT-4—such as incomplete coverage and prevalent anti-patterns—LLM Chaining offers a more solid and dependable approach to automatic unit test generation. Table 1 compares key automated unit test generation techniques, highlighting their strengths and limitations.

D. LIMITATIONS OF CURRENT APPROACHES IN APPLYING LLMS TO UNIT TEST GENERATION

Although LLM-based tools are very promising, there are several key limitations that need to be resolved. Most significant among these is generating non-compiling or semantically incorrect tests. According to a report in [30], 30% of tests generated by Codex did not compile due to problems such as missing imports, incorrect use of APIs, undefined variables, or mismatching signatures. Even if compilation passes, semantic bugs—such as wrong assertions or miscalculated logic—can provide a wrong impression of correctness.

Another significant limitation is lack of code coverage. For example, Fang et al. observed GPT-3 only covered 60% of branches in complex code, given to often neglect edge cases important for robustness [31]. Flawed practices—like redundant assertions or weak validation (e.g., assertTrue(true) or assertNotNull(result) without context)—also decrease test suite quality, as Kang et al. discussed [32].

Additionally, single-pass, static prompt-driven generation limits flexibility across various codebases and changing needs—a problem noted by Zhang et al. [33]. While partial remedies such as prompt engineering [34], [35] and post-processing regulations [32] exist, they tend to be ad hoc and incoherent.

These limitations bring ethical and security issues. Passing tests incorrectly can deceive developers, particularly in safety-critical use cases. Flawed tests included within CI/CD pipelines under lax control may cause production problems of a serious nature. The code generated could also accidentally include insecure patterns or break compliance standards, particularly in use cases with sensitive or regulated data.

To address those requirements, we introduce our LLM Chaining architecture—a multi-LLM sequential pipeline for improved syntactic and semantic correctness. In this pipeline, Gemini produces the first test, while GPT-4o refines and checks that test. This layered structure ensures improved alignment with an evolving codebase and results in higher-quality tests. Initial empirical findings on the HumanEval benchmark determined that LLM chaining obtains 99.05% branch coverage and 90.48% line coverage, which is better than single-model methods. This iterative improvement enables LLM chaining to adaptively cope with modular code modifications, effectively represent developer intent, and serve as an extensible and reliable approach to unit test automation.

III. METHODOLOGY

This section explains how large language models (LLMs) are used to automatically create Java test cases from start to end. The process focuses on using the complementary strengths of LLMs to automatically create high-quality test suites that use JUnit to test Java programs.

The Figure 1 shows the architecture diagram of the multi-stage pipeline, which shows how the proposed system will work as a whole.

The process consists of five main steps that work together to improve the quality and coverage of the test cases that are made while also making it easier to do the work by hand. These stages include: (1) Source Code Analysis, (2) Test Case Generation using Gemini, (3) Test Refinement using GPT-4o, (4) Final Test Validation, and (5) Coverage Measurement.

A. SOURCE CODE ANALYSIS

This stage focuses on extracting meaningful structural and semantic information from Java source code to guide effective test generation. It combines the use of a benchmark dataset with a parsing mechanism to identify and interpret code elements that influence how LLM prompts are constructed and test cases are generated. The subsections below describe the dataset used and the specific process of code analysis.

TABLE 1. Comparison of automated unit test generation techniques.

Technique	Strengths	Limitations
Symbolic Execution	High precision in exploring program paths; effective for C programs [18].	Path explosion in complex systems; difficulty with dynamic languages and SQL interactions [19].
SBST (e.g., EvoSuite)	Well-structured test cases; good branch coverage for Java programs [17].	High computational costs; limited adaptability to runtime changes [19].
Search-Based Testing	Effective at exploring large input spaces; adaptable to various optimization goals [25].	May get stuck in local optima; requires careful fitness function design [25].
Constraint-Based Testing	Precise input generation by solving path constraints; suitable for symbolic execution [26].	Computationally expensive; limited by constraint solver capabilities [26].
Concolic Testing	Combines concrete and symbolic execution for better path coverage [27].	Suffers from path explosion and struggles with loops or recursion [27].
Grammar-Based Testing	Suitable for structured input generation (e.g., parsers, compilers); grammar-guided fuzzing [28].	Requires manually defined grammars; less effective on non-structured inputs [28].
LLM-based Testing	Flexible, scalable, and context-aware test generation; supports dynamic environments [21].	Dependency on curated datasets; generalization issues across diverse codebases [22].

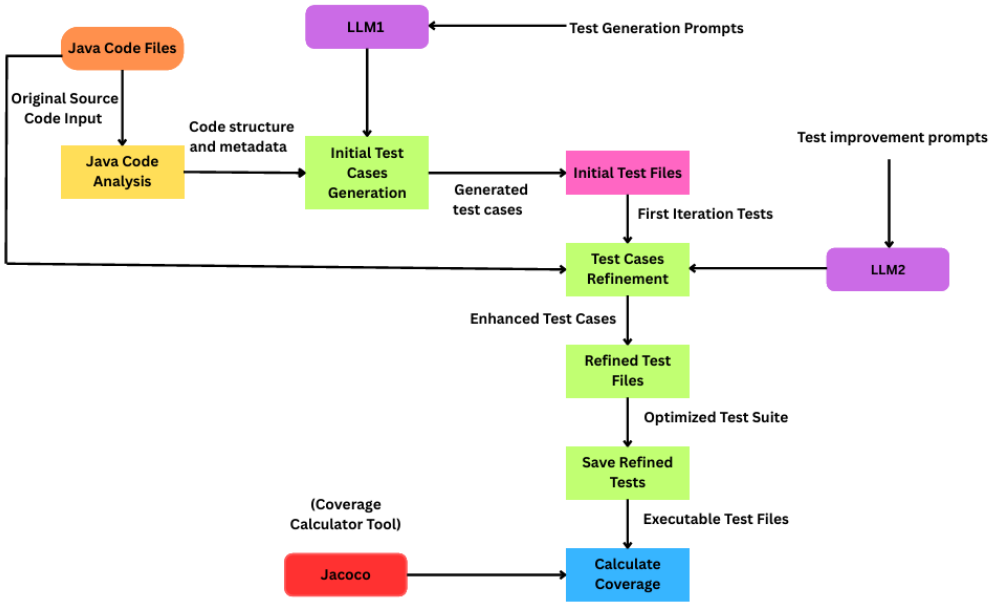

FIGURE 1. Automated java test case creation, Improvement, and coverage analysis using an LLM-chained framework.

1) DATASET: HUMANEVAL-JAVA

The dataset HumanEval from OpenAI consists of 164 programming problems with function signature, docstring, body, and some unit tests. They were all handwritten specifically so they would not be in the data for models that generate code. A sample from the HumanEval-Java dataset is shown in Figure 2, illustrating the structure and components of each class. One of the strongest aspects of the dataset is the evaluation plan using measurements of code coverage (using JaCoCo) and failure rate (using JUnit). All measures quantify syntactic correctness, function coverage, and handling of edge cases into an overall score of LLM performance tested in its ability to automate test generation. Richer metadata and docstrings will improve clarity of the prompt again for LLMs about the best and most efficient tests to generate.

2) SOURCE CODE ANALYSIS AND METADATA EXTRACTION

The process begins with parsing Java source files for structural and semantic information pertinent to test generation. This step is essential to guide the prompt creation step that follows. The parsing step involves reading each.java file, removing comments to avoid polluting the prompts, and utilizing regular expressions to identify the main program elements (for example, class declarations, public method declarations, and conditional statements) in the code. The goal of the extraction step is to determine:

- **Package name:** Helps create the suitable test class header.
- **Class name:** Used for the test class file's naming and comparison.

```java
package original;

class GreatestCommonDivisor {
    /**
     * Return the greatest common divisor of two integers a and b.
     * > greatestCommonDivisor(3, 5)
     * 1
     * > greatestCommonDivisor(25, 15)
     * 5
     */
    public static int greatestCommonDivisor(int a, int b) {
        if (a == 0) {
            return b;
        }
        return greatestCommonDivisor(b % a, a);
    }
}
```

FIGURE 2. The HumanEval dataset sample.

- **Method signatures:** It is helpful for monitoring the number of functions and input parameters that require verification.
- **Conditional branches (e.g., `if`, `switch`):** They act as substitutes for assessing branch complexity and guiding the number of test cases needed to ensure comprehensive coverage.
- **Data types:** To support realistic generation, primitive types (`int`, `double`, `boolean`, etc.) and objects are extracted from method parameters and variable declarations.

The metadata needed to create effective LLM prompts that identify control flow structures and type constraints within the source class is provided by this extraction. The system can prevent the creation of test cases for private and inaccessible methods by using this first-step analysis, which improves the generated tests semantic relevance. The process of extracting metadata from a Java class is explained in Algorithm 1.

B. TEST CASE GENERATION USING GEMINI API

Gemini is instructed to generate an comprehensive JUnit test class. Because of Gemini's exceptional ability to produce lengthy, cohesive, and structurally sound code completions—particularly for Java test cases—we chose it as the first test generator. Creating lengthy, multi-method JUnit test classes from the raw source code metadata was a good fit for Gemini. Gemini's high token efficiency and ability to follow instructions were valued by us. At this point in the pipeline, its ability to efficiently generate correctly formatted code in a single pass is essential to maintaining the output's syntactic and semantic integrity and avoiding premature truncation. Using the structural metadata from the previous step, this step generates a prompt asking the Gemini model to produce particular, syntactically correct test classes. We started with very basic instructions like "generate JUnit test cases," but we ran into problems with incomplete code, skipped edge cases, and private methods that shouldn't have been tested. In order to address this, we began adding more information to the prompts, like method names, the quantity of conditions, and data types, which improved the model's comprehension of the code. The main goal is to use applicable and realistic inputs to exercise all of the executable lines and conditional branches. The generated test code must not contain private methods, adhere to JUnit standards, and work with the original Java class. The following should be the prompt strategy:

- Use complete Java source code within triple backticks for syntax readability.
- State the exact number of branches (estimated from if and switch statements).
- State all data types used in the class that are relevant (int, boolean, String, etc.).
- Tell the model to generate test cases for all paths of execution, edge cases (zero, negative, and boundary values), and invalid input scenarios.
- Demand significant assertions and good method names, and do not truncate test code by telling it to include all closing braces explicitly.

Algorithm 2 illustrates the generation of initial test cases using the Gemini model.

C. TEST REFINEMENT USING GPT-4O

To enhance the original test suite, GPT-4o is called upon to insert edge cases, refine input diversity, and correct logical test defects. It was introduced in the refinement phase due to its strong contextual understanding of both code and accompanying test logic, rich semantic reasoning, and high accuracy in identifying edge cases and assertion depth. Rather than starting with GPT, which could risk over-engineering test cases or emphasizing correctness prematurely (before broad coverage was attempted), we chose to invoke GPT-4o only after a structured base was built by Gemini. This made it possible for GPT to concentrate only on refinement—improving correctness, input diversity, and coverage—rather than structural generation. Both the original class and the derived test class are included in the GPT-4o prompt, along with an extensive collection of refinement commands. The main changes that are being asked for are

- Add tests for corner and boundary cases (such as null values, zero, and max/min bounds).
- Increase the depth of the assertion by verifying more precise return values and post-conditions.
- To maintain encapsulation boundaries, avoid creating test cases for private methods.
- Test methods should be properly named and mapped to the methods that are being tested.
- To expand the scope of test coverage, incorporate realistic input variations.
- Make sure the test suite compiles and executes properly to confirm accuracy.

GPT-4o refines the test cases with a variety of inputs and assertions, as explained in Algorithm 3.

Algorithm 1 Java Class Metadata Extraction

Require: Java source file `java_file`
Ensure: `metadata` dictionary {package, class_name, method_signatures, conditions, data_types}

```
content ← read_file(java_file)
package_name ← extract using regex r"package\s+(.*?);"
class_name ← extract using regex r"(public\s+)?(class|interface|enum)\s+(\w+)"
method_signatures ← find all matches of pattern r"(mod)?\s+ret\s+name\s*(.*?)\s*{"   ▷ mod = access modifier, ret = return type
conditions ← find all matches of r"if\s*\((.*?)\)"
data_types ← regex match for types (int, double, float, ...) and variable names
data_types ← find all matches of r"\b(type)\b\s+(\w+)"   ▷ type = int, double, ...
metadata ← {package: package_name, class_name: class_name, method_signatures: method_signatures, conditions: conditions, data_types: data_types}
return metadata
```

Algorithm 2 Generate Test Cases With Gemini

Require: Java source code `java_code`, class metadata `class_info`
Ensure: `test_code` (raw JUnit test class as String)

```
prompt ← construct_prompt(java_code, class_info)   ▷ Combine source and metadata into structured prompt
response ← HTTP_POST(Gemini_API_URL, prompt)
raw_output ← extract_text(response)
cleaned_output ← clean_code_response(raw_output)
if cleaned_output is null then
    retry_prompt ← construct_simplified_prompt(java_code)
    response ← HTTP_POST(Gemini_API_URL, retry_prompt)
    raw_output ← extract_text(response)
    cleaned_output ← clean_code_response(raw_output)
end if
return cleaned_output
```

Algorithm 3 Refine Test Cases With GPT-4o

Require: Java source code `java_code`, raw test class `test_code`, class metadata `class_info`
Ensure: `refined_test_code` (Refined JUnit test class)

```
gpt4o_prompt ← build_prompt(java_code, test_code, class_info)
response ← HTTP_POST(GPT4O_API_URL, gpt4o_prompt)
cleaned_output ← clean_code_response(response)
if cleaned_output is null then
    retry_prompt ← build_simplified_prompt(...)
    response ← HTTP_POST(GPT4O_API_URL, retry_prompt)
    cleaned_output ← clean_code_response(response)
end if
return cleaned_output
```

D. TEST CLASS WRITING AND EXECUTION

To conform to the Java project structure, refined test classes are written to the relevant test directory. The pattern `{ClassName}Test.java` is used when naming files, and directory layers will be created if none already exist. By verifying the file creation process and the file path, the logging mechanism makes sure that every test class is tracked and positioned correctly. Build tools such as Maven or Gradle automatically compile and run the tests after they are written. Any test failures, including compilation errors, failed assertions, or unexpected exceptions, are automatically identified and recorded during this phase. The procedure for creating the improved test cases is described in Algorithm 4.

E. COVERAGE MEASUREMENT

In order to assess the effectiveness of the generated tests, we integrate a reliable coverage measurement tool into the testing pipeline. This process is crucial for assessing how thorough test runs are, and is used to identify areas

Algorithm 4 Write Refined Test Class

Require: Class name `class_name`, refined test code `test_code`
Ensure: `file_path` (Saved test file path)

```
file_path ← TEST_DIR + "/" + class_name + "Test.java"
if TEST_DIR does not exist then
    create_directory(TEST_DIR)
end if
write_file(file_path, test_code, encoding="UTF-8")
return file_path
```

that require further work. Coverage analysis tools are integrated into the process, leading to a feedback-oriented improvement cycle that guarantees LLM-generated tests are comprehensive as well as syntactically correct. The next subsection describes JaCoCo, the coverage measurement tool providing evidence to our pipeline for testing generated test suites effectiveness and improving the generated test suites iteratively.

1) JACOCO FOR COVERAGE MEASUREMENT

The Java code coverage library is called JaCoCo. It explains the amount of source code that runs when testing an application and provides information about the completeness of test cases generated. JaCoCo verifies line coverage by verifying how many lines of code have been run during testing. It also verifies branch coverage to make sure that conditional paths have been tested (if/else/switch statements). It verifies instruction coverage via executed bytecode instructions and inspected complexity, quantified as cyclomatic complexity, which is the number of independent paths within the program. It offers detailed statistics on line coverage (the rate of lines covered by tests) and branch coverage (the rate of decision points tested). The integration is generally managed using Maven or Gradle plugins, which are set to gather coverage information during the test phase of the build lifecycle. When run, JaCoCo generates comprehensive reports in HTML and XML formats. These reports are automatically processed to identify if 100% line and branch coverage goals are achieved. When gaps in coverage are identified, the related methods or branches are indicated, and the steps of prompt generation or test refinement are redone accordingly.By enabling data-driven enhancement of test prompts and guiding LLMs toward producing more thorough and accurate test suites, the feedback loop transforms JaCoCo into a validation checkpoint of significance.

Please see Appendix for comprehensive details on model versions and prompt templates.

IV. RESULTS AND ANALYSIS

We provide a thorough analysis of the suggested LLM-based JUnit test generation techniques in this section. In order to evaluate the efficacy of single and chained LLMs in generating high-quality and fault-tolerant test cases, we first compare their outputs in terms of compilation status, correctness, coverage, and mutation testing. After that, we highlight coverage differences by comparing LLM-based approaches with a conventional non-LLM baseline (Randoop). Finally, to put our findings in context, we address possible threats to validity and explain why the Gemini + GPT-4o combination performs better than other configurations.

TABLE 2. Compilation status of the generated unit tests.

Model	Errors	Total Tests	Compilability (%)
Gemini	13	3,077	99.58
GPT-4o	18	2,454	99.27
Claude-3.5 Sonnet	22	2,339	99.06
Gemini + GPT-4o	24	3,508	99.32
Gemini + Claude-3.5	35	3,534	99.01

A. COMPILATION STATUS

Tests that failed to compile because of syntax errors, missing imports, unresolved types, or malformed code blocks were counted to determine the compilation status. A failure here refers specifically to compile-time errors, while runtime issues were excluded from this metric. Gemini showed the highest compilation success with only 13 errors out of 3077 test cases (99.58%), followed by GPT-4o with 18 errors out of 2454 tests (99.27%) and Claude-3.5 Sonnet with 22 errors out of 2339 tests (99.06%), single LLMs showed high compilation success rates. Strong compilation performance was also maintained by combined LLMs; Gemini + GPT-4o generated 24 errors out of 3508 tests (99.32%), and Gemini + Claude-3.5 Sonnet generated 35 errors out of 3534 tests (99.01%). All approaches achieved over 99% compilation success, indicating strong performance in generating syntactically correct test code, even if combined LLMs generated more test cases and slightly increased errors.

Compilation success across different LLM configurations is visualized in Figure 3, while Table 2 summarizes the exact number of errors and total tests per model. Gemini achieves the highest success rate, followed by GPT-4o and the Gemini + GPT-4o combination. Claude-3.5 Sonnet and the Gemini + Claude-3.5 pipeline show comparatively lower success rates.

B. TEST CORRECTNESS

Correctness was measured with respect to runtime test failures that manifested when test cases ran without

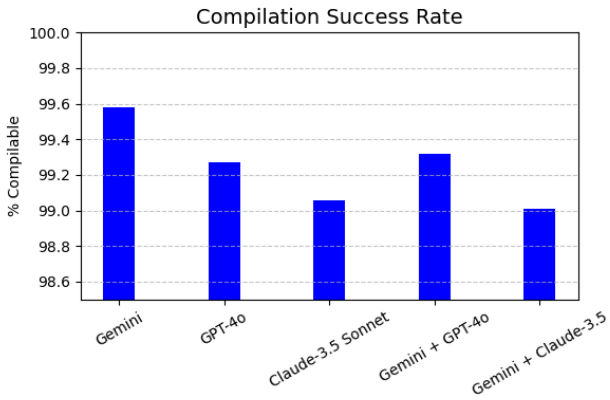

FIGURE 3. Compilation success rates (% compilability) for different LLM configurations.

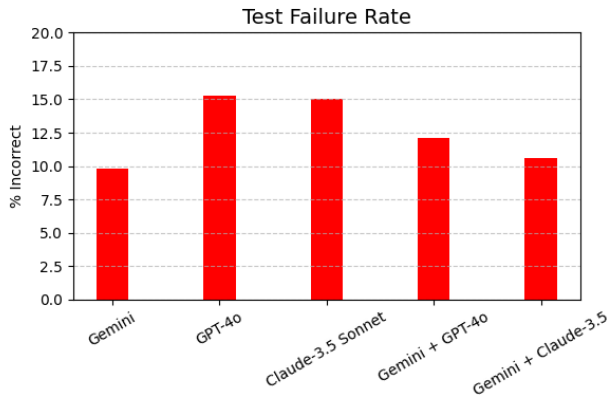

FIGURE 4. Test failure rates for various LLM configurations (% incorrect outputs).

compile-time errors but failed because of logical inconsistency, invalid assertion, or inadequate input coverage. These are semantic failures—i.e., the test logic fails to accurately represent the intended behavior of the original function. Among the individual LLMs, Gemini recorded the lowest number of failures (301/3077 tests, 9.78% failure rate), followed by Claude-3.5 Sonnet (351/2339, 15.01%) and GPT-4o (375/2454, 15.28%). Compared to this, the combined LLMs were also more accurate in nature, with Gemini + Claude-3.5 Sonnet recording 375 failures in 3534 tests (10.61%) and Gemini + GPT-4o recording 426 failures in 3508 tests (12.14%). The results show that the combination of LLMs can improve test correctness, especially in being able to handle more varied and larger test cases, although performance still depends on the specific combination used.

Gemini + GPT-4o has a marginally larger runtime failure rate than other LLM pairings since the pipeline produces a larger variety of test cases. Based on method signatures, Gemini initializes well-structured test skeletons, and GPT-4o refines them with a broader variety of inputs, edge cases, and complex assertions. Semantic failures—tests that compile but fail at runtime from logical errors or from lack of complete input handling—are more probable as well. Thus, instead of being a measure of failure to generate tests, the increased failure rate is a measure of creating richer, more comprehensive tests.

Even though these results show variations in output accuracy between models, we must determine how much the test suites exercise the code and how accurately they identify errors in order to evaluate the generated test suites. We concentrate on test coverage and mutation testing to allow for a more thorough comparison.

Figure 4 illustrates variations in output correctness by showing the percentage of failed test cases for different LLM configurations. Table 3, which shows the proportion of incorrect outputs and failed test cases for each model, provides additional support for this data.

TABLE 3. Percentage of incorrectly generated tests.

Model	Failures	Total Tests	Incorrect (%)
Gemini	301	3,077	9.78
GPT-4o	375	2,454	15.28
Claude-3.5 Sonnet	351	2,339	15.01
Gemini + GPT-4o	426	3,508	12.14
Gemini + Claude-3.5	375	3,534	10.61

C. TEST COVERAGE

Instruction, branch, line, and complexity coverage were the four main metrics used to evaluate test coverage. Compared to single LLMs, combined LLMs had better coverage in every coverage area. The best overall coverage was obtained by the Gemini + GPT-4o combined models, which had 93.91% instruction coverage, 99.05% branch coverage, 90.48% line coverage, and 82.52% complexity coverage. The Gemini + Claude-3.5 Sonnet combined models had 93.85% instruction coverage, 98.66% branch coverage, 90.43% line coverage, and 82.11% complexity coverage. GPT-4o had the highest overall coverage of all the individual LLM models, with 92.76% coverage for instructions, 96.84% for branches, 89.68% for lines, and 80.18% for complexity. This implies that by increasing coverage and simultaneously reaching the maximum number of tests, the use of combined LLMs can result in more thorough and comprehensive testing.

Test coverage comparisons between different LLM models are shown in Figure 5, and the precise numbers for instruction, branch, line, and complexity coverage are shown in Table 4. In addition, Figure 6, Figure 7, Figure 8, and Figure 9 show both single and combined model configurations and show line coverage trends across individual classes. With the highest overall coverage, especially in branch and line metrics, the Gemini + GPT-4o and Gemini + Claude-3.5 combinations demonstrate enhanced test suite efficacy.

D. MUTATION TESTING

Mutation testing was conducted using the PIT framework [36] to assess the fault-detection ability of the

TABLE 4. Coverage results of the generated tests.

Model	Instruction Coverage (%)	Branch Coverage (%)	Line Coverage (%)	Complexity Coverage (%)
Gemini	91.18	95.18	87.89	79.67
GPT-4o	92.76	96.84	89.68	80.18
Claude 3.5 Sonnet	91.35	94.23	88.29	78.56
Gemini + GPT-4o	**93.91**	**99.05**	**90.48**	**82.52**
Gemini + Claude 3.5	93.85	98.66	90.43	82.11

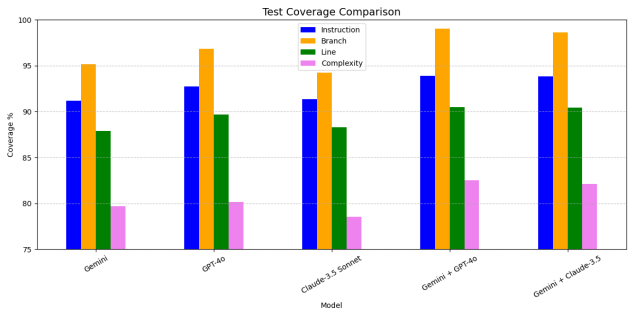

FIGURE 5. Comparison of test coverage metrics across various LLM configurations. Metrics include instruction, branch, line, and complexity coverage.

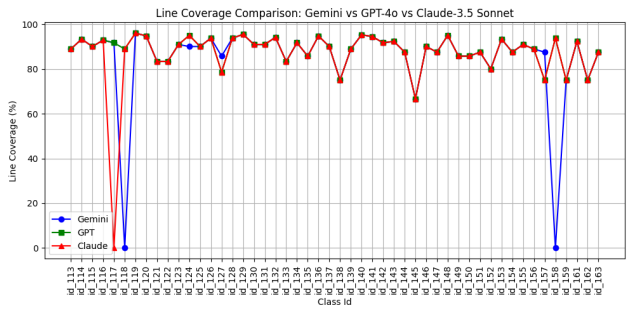

FIGURE 6. Line coverage comparison across individual classes for Gemini, GPT-4o, and Claude-3.5 Sonnet.

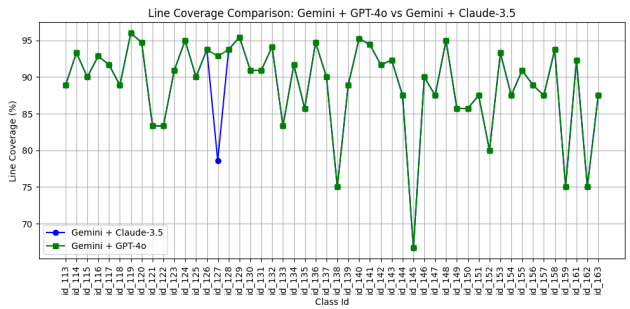

FIGURE 7. Line coverage comparison across individual classes for combined model strategies: Gemini + GPT-4o and Gemini + Claude-3.5.

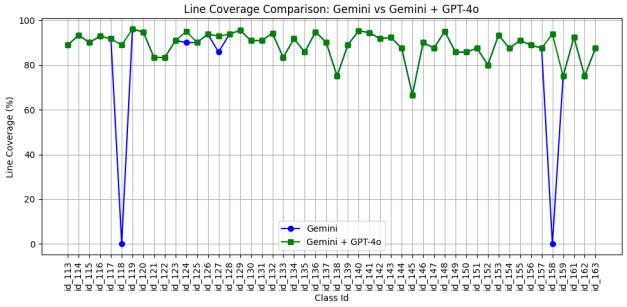

FIGURE 8. Line coverage comparison across individual classes for Gemini and Gemini + GPT-4o.

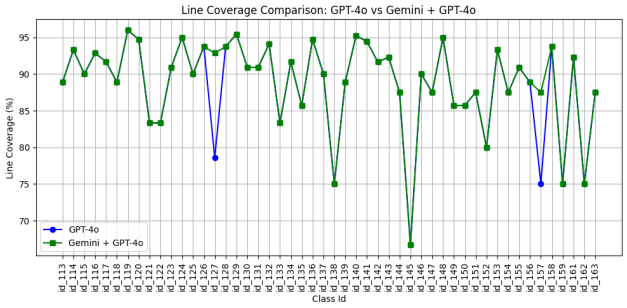

FIGURE 9. Line coverage comparison across individual classes for GPT-4o and Gemini + GPT-4o.

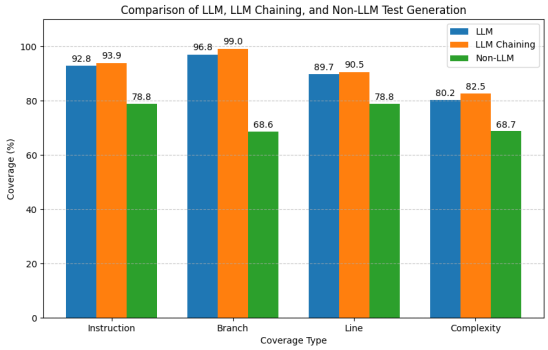

FIGURE 10. Comparison of instruction, branch, line, and complexity coverage for LLM, LLM chaining, and non-LLM approaches.

generated test suites. Unlike line or branch coverage, which primarily measure execution of code paths, mutation testing evaluates the effectiveness of tests in identifying artificially injected faults, known as mutants. A higher mutation coverage indicates that the test suite is capable of detecting a greater proportion of faults, thereby reflecting stronger fault detection capability and higher overall test quality.

Table 5 summarizes the mutation testing results across different LLM configurations Among the individual models, GPT-4o achieved the highest mutation coverage of 93.00%, followed closely by Gemini of 91.74% and Claude-3.5 Sonnet of 90.67%. The combined LLM approaches further improved performance: Gemini + GPT-4o attained the maximum coverage of 94.32%, with Gemini + Claude-3.5 achieving 94.26%. These findings imply that collaborative multi-model generation generates more robust and resilient tests than single-model approaches, as LLM

Chaining improves the fault-detection capability of generated test cases. All things considered, the results show how well using several LLMs can increase mutation coverage and fortify test quality.

E. COVERAGE COMPARISON BETWEEN LLM AND NON-LLM APPROACHES

Randoop was used as the standard for conventional automated test generation [38]. 78.82% instruction coverage, 68.64% branch coverage, 78.84% line coverage, and 68.70% complexity coverage were attained by the test cases produced by Randoop.

By contrast, LLM-based methods perform noticeably better than Randoop on every coverage metric. Claude-3.5 Sonnet, Gemini, and GPT-4o were among the individual models that achieved the following: complexity coverage from 78.56% to 80.18%, branch coverage from 94.23% to 96.84%, line coverage from 87.89% to 89.68%, and instruction coverage between 91.18% and 92.76%.

Performance was further enhanced by LLM Chaining techniques. Gemini + GPT-4o achieved similar results as Gemini + Claude-3.5, with 93.91% instruction coverage, 99.05% branch coverage, 90.48% line coverage, and 82.52% complexity coverage.

Table 6 summarizes these findings by presenting the highest coverage values for each approach, while Figure 10 illustrates the comparison of the various approaches. These results show that LLM-based test generation, especially with LLM Chaining, is a more efficient way to create thorough and reliable JUnit test suites because it offers substantially more coverage than conventional non-LLM techniques like Randoop.

F. WHY GEMINI COMBINED WITH GPT-4O PRODUCES BETTER RESULTS

We hypothesize that the combined Gemini + GPT-4o pipeline benefits from a division of responsibilities. Gemini, guided by structural metadata, may be more effective at generating organized and syntactically correct test skeletons aligned with method signatures and control structures. Applying GPT-4o later could improve the tests' semantic quality by enhancing input diversity, exception handling, and edge case coverage. Together, this "coarse-to-fine" cascade may allow Gemini to construct the initial skeleton while GPT-4o refines it, potentially identifying missing assertions, improving input-output mappings, and strengthening post-condition validation.

While Gemini alone produces the lowest test failure rate (9.78%), the combined Gemini + GPT-4o approach achieves significantly higher coverage across instruction (93.91%), branch (99.05%), line (90.48%), and complexity (82.52%). The slight increase in failure rate (12.14%) reflects the generation of more challenging and diverse test cases, which in turn improves the ability to expose hidden bugs and exercise under-tested code paths. Thus, when balancing breadth of coverage and defect detection potential, multi-LLM chaining represents the more effective approach, even if single LLMs achieve marginally lower failure rates.

G. THREATS TO VALIDITY

In this subsection, we state possible risks that could compromise the dependability, applicability, and explainability of our multi-LLM chaining technique. Apart from prompt sensitivity and dataset constraints, other LLM uncertainty, code complexity, and computation overhead-related considerations are discussed.

The initial threat is the overreliance on prompt engineering and deterministic chaining logic employed in orchestrating cross-interaction among multiple LLMs. Although the chaining approach starting with Gemini for test generation and improvement by GPT-4o provided better outcomes, the influence of prompt wording and order on quality of output is still a risk. Small prompt variations or task order deviations can impact the reproducibility and reliability of test cases.

A second threat arises due to the unpredictability and stochastic nature of LLMs. We note that even at low temperature, the same prompt may give slightly different outputs at different runs, and such variation may cascade into the chained process. Randomness cannot be completely eliminated, although several independent runs were performed to mitigate the effect of randomness on the output.

Another threat is in the use of the HumanEval dataset for benchmarking purposes. While it offers standardized functions appropriate for test generation, it is primarily made up of algorithmic problems that neither capture fully the diversity nor the complexity of commercial software projects. Larger classes, with more complex control flow, state, and dependencies are also likely to reduce the effectiveness of our approach, especially when prompt truncation or context-window limits are encountered.

Another possible threat relates to the measure of code coverage as a metric. We measured code coverage with the tool JaCoCo, which tracks both line coverage and branch coverage. However, high coverage does not equate to good testing and bug detection. Without the use of fault injection or mutation testing, we do not know if the functional adequacy of the tests that were generated.

Another practical limitation is that a multi-LLM workflow can be computationally expensive. Chaining Gemini, GPT-4o adds additional latency and API usage costs. This overhead is fine for controlled experiments, but may prove too large for integration into real CI/CD pipelines, unless caching or incremental regeneration approaches are implemented.

Lastly, the application of LLMs was somewhat complicated. While chaining and merging reduced redundancy while improving coverage, there is a risk of overfitting to certain evaluation metrics (e.g., coverage %) instead of real behavioral correctness. In addition, manual merging of LLMs, to my knowledge, might not be generalizable or scalable across situations.

TABLE 5. Mutation coverage results of the generated tests.

Model	Total Mutations	Mutations Killed	Mutations Survived	Mutation Coverage (%)
Gemini	1,587	1,456	131	91.74
GPT-4o	1,587	1,476	111	93.00
Claude-3.5 Sonnet	1,587	1,439	148	90.67
Gemini + GPT-4o	1,587	1,497	90	94.32
Gemini + Claude-3.5	1,587	1,496	91	94.26

TABLE 6. Comparison of test coverage for LLM, LLM chaining, and Non-LLM approaches.

Approach	Instruction Coverage (%)	Branch Coverage (%)	Line Coverage (%)	Complexity Coverage (%)
LLM (GPT-4o)	92.76	96.84	89.68	80.18
LLM Chaining (Gemini + GPT-4o)	93.91	99.05	90.48	82.52
Non-LLM (Randoop)	78.82	68.64	78.84	68.70

TABLE 7. Summary of LLMs and baselines.

LLM Model	Provider	Version / Model ID	Inference Params	Access Date (Month-Year)	Prompt Template / Usage Summary
Gemini	Google	gemini-2.0-flash	Temperature=0.2, Top-p=0.9	August 2025	Generates initial JUnit test skeletons from Java source metadata (class name, methods, conditions, data types).
GPT-4o	OpenAI	GPT-4o	Temperature=0.3, Top-p=1.0	August 2025	Refines Gemini-generated test cases by adding edge cases, improving assertions, and ensuring code correctness.
Claude-3.5 Sonnet	Anthropic	claude-3.5-sonnet	Temperature=0.3, Top-p=1.0	August 2025	Refines Gemini-generated test cases by adding edge cases, improving assertions, and ensuring code correctness.
Randoop (Baseline)	University of Washington	4.3.4	N/A	September 2025	Used as a non-LLM baseline for automated test generation comparison. Command: `java -classpath randoop-all-<version>.jar randoop.main.Main gentests -testclass=<ClassName> -time-limit=<seconds>`

- **Internal Validity:** Potential sources of bias are introduced by the LLM chaining workflow design. The format and content of the prompts provided to each LLM may have an impact on the caliber and comprehensiveness of the unit tests that are produced. Variable results can be produced by slight changes in wording or instruction sequence, which jeopardizes the consistency and dependability of the results.
- **External Validity:** This study's conclusions are restricted to the HumanEval dataset, Java, and JUnit. The results applicability to different testing frameworks, programming languages, or domain-specific software systems is restricted by these limitations. The relevance of LLM chaining to actual industrial codebases and larger programming ecosystems should be investigated in future studies.
- **Construct Validity:** Generated test evaluation was done on the basis of code coverage, which, although popular, cannot completely represent test quality or fault detection efficacy. Moreover, since there were no manual oracles, it was hard to tell if edge cases were actually tested. We somewhat alleviated this by manually inspecting a portion of test cases and requiring coverage consistency across repeated runs.
- **Conclusion Validity:** The findings, although encouraging, are on the basis of a limited set of test cases (3,508 runs) and without stat-testing for significance. We thus cannot be very confident that the improvements we notice are not due to random chance. Although LLM chaining performance was better than single-LLM configurations uniformly, a replication study using other datasets and models is needed to confirm the stability of our results.

V. CONCLUSION AND FUTURE WORK

This paper shows how the automation of test case generation for Java programs can be greatly improved by integrating several large language models (LLMs) in a structured pipeline. In our approach, coverage is the primary measure of test quality, as it remains the most comprehensive standard within software testing research and practice. To highlight the strength of proposed work, we also integrated error detection methods such as mutation testing, which evaluates whether the generated test suites can effectively detect and eliminate the injected faults or not, which provided the greater insight into test robustness and effectiveness. Our finding suggests that Gemini + GPT-4o is the top-performing model which obtained better results in both coverage measures and mutation testing. So, it confirms the reliability of Gemini + GPT-4o in real-world applications. These results verified the industrial relevance of LLM chaining to enhance software testing practice and future industrial deployment potential.

The JaCoCo usage for coverage measurement and JUnit for test execution further validates the technical integrity of the generated test code. Although traditional benchmarks like HumanEval are useful for testing, their shortcomings—particularly in the case of algorithmic Java and function-level evaluation emphasize the need to measure coverage depth and generality. To meet broader enterprise requirements, our multi-LLM approach can be easily extended to database, API, and user interface applications that may show its applicability to various real-world situations beyond standard datasets.

In the future, integration of LLM-driven testing frameworks in CI/CD pipelines will enable teams to test scalability and it will help in analyzing time cost trade-offs upon supporting developments at the enterprise level. Significant industrial initiatives suggest that requirement for adaptive models, enabling enhanced efficiency and dependability as testing requirements increase. Chaining multiple commercial APIs can result in noticeable computation overhead because of network latency, model inference time, and sequential pipeline execution. Every API call also has a monetary cost, which can add up rapidly if one is generating lots of tests, or doing frequent runs in a CI/CD pipeline. As a result, the overall approach must balance accuracy with operational efficiency. In real-world deployments, organizations should consider batch requests, cache intermediate results, or trigger the pipeline at a granularity only involving changed modules, in order to keep both latency and cost manageable. Further research will examine dynamic LLM selection methods in terms of source code complexity, and adaptive prompt tuning methods based on past performance and ongoing feedback. These enhancements have the potential to render LLM-based test generation a flexible, usable part of modern software engineering.

Overall, this work makes a strong and practical case for adopting AI in software testing by addressing real industry needs and integrating mutation-based error detection. It overlays the way for more reliable, efficient, and human-centric development practices that evolve with technological change.

APPENDIX

SUMMARY INFORMATION ON LLMS AND BASELINES

This appendix provides an overview of the large language models (LLMs) and baseline methods used in this study. The summarized details include provider, model version, access date, and prompt usage. This information enhances reproducibility and transparency of our experiments.

The summarized information is presented in Table 7. The complete set of prompt templates and model versions used in our experiments are publicly available at: https://github.com/ushasri3110/LLMChaining

REFERENCES

[1] L. Tripp, *IEEE Standards Collection: Software Engineering*. New York, NY, USA: Institute of Electrical and Electronics Engineers, 1994.

[2] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed., Hoboken, NJ, USA: Wiley, 2011.

[3] A. Bertolino, "Software testing research: Achievements, challenges, dreams," in *Future of Software Engineering (FOSE)*. Minneapolis, MN, USA, 2007, pp. 85–103, doi: 10.1109/FOSE.2007.25.

[4] S. Saeed, N. Z. Jhanjhi, M. Naqvi, and M. Humayun, "Analysis of software development methodologies," *Int. J. Comput. Digit. Syst.*, vol. 8, no. 5, pp. 445–460, 2019.

[5] Strategic Planning, *The Economic Impacts of Inadequate Infrastructure for Software Testing*. Gaithersburg, MD, USA: National Institute of Standards and Technology, vol. 1, 2002.

[6] D. Kumar and K. K. Mishra, "The impacts of test automation on Software's cost, quality and time to market," *Procedia Comput. Sci.*, vol. 79, pp. 8–15, Apr. 2016.

[7] S. Alagarsamy, C. Tantithamthavorn, and A. Aleti, "A3Test: Assertion-augmented automated test case generation," *Inf. Softw. Technol.*, vol. 176, Dec. 2024, Art. no. 107565.

[8] K. Pei, Y. Cao, J. Yang, and S. Jana, "DeepXplore: Automated whitebox testing of deep learning systems," in *Proc. 26th Symp. Operating Syst. Princ.*, 2017, pp. 1–18.

[9] L. Yang, C. Yang, S. Gao, W. Wang, B. Wang, Q. Zhu, X. Chu, J. Zhou, G. Liang, Q. Wang, and J. Chen, "On the evaluation of large language models in unit test generation," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2024, pp. 1607–1619.

[10] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024.

[11] Q. Wang, J. Wang, M. Li, Y. Wang, and Z. Liu, "A roadmap for software testing in open-collaborative and AI-powered era," *ACM Trans. Softw. Eng. Methodology*, vol. 34, no. 5, pp. 1–17, Jun. 2025.

[12] M. R. Lyu, B. Ray, A. Roychoudhury, S. H. Tan, and P. Thongtanunam, "Automatic programming: Large language models and beyond," *ACM Trans. Softw. Eng. Methodology*, vol. 34, no. 5, pp. 1–33, Jun. 2025.

[13] C. Kumar, D. K. Yadav, and M. Prasad, "Predicting fault-prone software modules using Bayesian belief network: An empirical study," *Int. J. Syst. Assur. Eng. Manag.*, vol. 16, pp. 2204–2218, Nov. 2025, doi: 10.1007/s13198-025-02809-1.

[14] A. Lops, F. Narducci, A. Ragone, M. Trizio, and C. Bartolini, "A system for automated unit test generation using large language models and assessment of generated test suites," in *Proc. IEEE Int. Conf. Softw. Test., Verification Validation Workshops (ICSTW)*, Mar. 2025, pp. 29–36.

[15] Y. Li, Y. Zhong, L. Yang, Y. Wang, and P. Zhu, "LLM-guided crowdsourced test report clustering," *IEEE Access*, vol. 13, pp. 24894–24904, 2025.

[16] T. Godage, S. Nimishan, S. Vasanthapriyan, V. Palanisamy, C. Joseph, and S. Thuseethan, "Evaluating the effectiveness of large language models in automated unit test generation," in *Proc. 5th Int. Conf. Adv. Res. Comput. (ICARC)*, Feb. 2025, pp. 1–6.

[17] G. Fraser and A. Arcuri, "A retrospective on whole test suite generation: On the role of SBST in the age of LLMs," *IEEE Trans. Softw. Eng.*, vol. 51, no. 3, pp. 874–878, Mar. 2025.

[18] S. Genç, "Automatic unit test generator software by collecting run-time data," *Sigma J. Eng. Natural Sci.*, vol. 42, no. 4, pp. 988–1008, 2024.

[19] S. Vogl, S. Schweikl, G. Fraser, A. Arcuri, J. Campos, and A. Panichella, "Evosuite at the SBST 2021 tool competition," in *Proc. IEEE/ACM 14th Int. Workshop Search-Based Softw. Test. (SBST)*, May 2021, pp. 28–29.

[20] B. Vogel-Heuser, A. Fay, I. Schaefer, and M. Tichy, "Evolution of software in automated production systems: Challenges and research directions," *J. Syst. Softw.*, vol. 110, pp. 54–84, Dec. 2015.

[21] H. Keuning, J. Jeuring, and B. Heeren, "A systematic literature review of automated feedback generation for programming exercises," *ACM Trans. Comput. Educ.*, vol. 19, no. 1, pp. 1–43, Mar. 2019.

[22] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[23] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.

[24] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, Apr. 2024.

[25] B. Patel, P. Gagrani, S. Sheth, K. Thakur, P. Varahan, and S. Shim, "StarryStudioAI: Automating UI design with code-based generative AI and RAG," in *Proc. IEEE 15th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2025, pp. 00937–00944.

[26] P. McMinn, "Search-based software test data generation: A survey," *Softw. Test., Verification Rel.*, vol. 14, no. 2, pp. 105–156, Jun. 2004, doi: 10.1002/stvr.294.

[27] K. Sen, D. Marinov, and G. Agha, "CUTE: A concolic unit testing engine for c," in *Proc. 10th Eur. Softw. Eng. Conf. 13th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, Sep. 2005, pp. 263–272, doi: 10.1145/1081706.1081750.

[28] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed automated random testing," in *Proc. ACM SIGPLAN Conf. Program. Lang. design Implement.*, Jun. 2005, pp. 213–223, doi: 10.1145/1065010.1065036.

[29] P. Godefroid, H. Peleg, and R. Singh, "Learn&fuzz: Machine learning for input fuzzing," in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Oct. 2017, pp. 50–59, doi: 10.1109/ASE.2017.8115618.

[30] M. L. Siddiq, J. C. S. Santos, S. Devareddy, and A. Müller, "SALLM: Security assessment of generated code," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng. Workshops*, Oct. 2024, pp. 54–65.

[31] J. Fan, Y. Ai, X. Liu, Y. Deng, and Y. Li, "Coding latent concepts: A human and LLM-coordinated content analysis procedure," *Commun. Res. Rep.*, vol. 41, no. 5, pp. 324–334, Oct. 2024.

[32] J. Kang, S. Kim, J. Kim, E. Lee, and D. Oh, "Automatic evaluation of hallucination effects in knowledge-based dialogue using large language model," in *Proc. Annu. Conf. Human Lang. Technol.*, 2024, pp. 566–569.

[33] Q. Zhang, W. Sun, C. Fang, B. Yu, H. Li, M. Yan, J. Zhou, and Z. Chen, "Exploring automated assertion generation via large language models," *ACM Trans. Softw. Eng. Methodology*, vol. 34, no. 3, pp. 1–25, Mar. 2025.

[34] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt injection attack against LLM-integrated applications," 2023, *arXiv:2306.05499*.

[35] Q. Yang, Z. Wang, H. Chen, S. Wang, Y. Pu, X. Gao, W. Huang, S. Song, and G. Huang, "PsychoGAT: A novel psychological measurement paradigm through interactive fiction games with LLM agents," 2024, *arXiv:2402.12326*.

[36] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "PIT: A practical mutation testing tool for Java (demo)," in *Proc. 25th Int. Symp. Softw. Test. Anal.*, Jul. 2016, pp. 449–452, doi: 10.1145/2931037.2948707.

[37] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proc. 19th ACM SIGSOFT Symp. 13th Eur. Conf. Found. Softw. Eng.*, Szeged, Hungary, Sep. 2011, pp. 416–419, doi: 10.1145/2025113.2025179.

[38] C. Pacheco and M. D. Ernst, "Randoop: Feedback-directed random testing for Java," in *Proc. OOPSLA Companion*, 2007, pp. 815–816, doi: 10.1145/1297846.1297902.

CHANDAN KUMAR received the Ph.D. degree in computer science and engineering from the National Institute of Technology (NIT), Jamshedpur, India, in 2018. He is currently an Assistant Professor (Sl.Gr.) with the School of Computing, Amrita Vishwa Vidyapeetham, Amaravati Campus, India. He has published several journals and conference papers that are published in reputed journals and conference proceedings, such as IEEE, Elsevier, and Springer. His research interests include software engineering, machine learning/deep learning in predictive analysis, fuzzy logic, artificial intelligence, and scientometric analysis.

USHA SRI PONAKA is currently pursuing the B.Tech. degree in CSE-AI with Amrita Vishwa Vidyapeetham, Amaravati Campus, India. She has completed good projects in the area of predictive analysis in the health and software engineering domains. She is passionate about doing major predictive analysis work using artificial intelligence techniques to solve real-world problems.

PULA V. LAKSHMI NARASIMHA NAIDU is currently pursuing the B.Tech. degree in CSE-AI with Amrita Vishwa Vidyapeetham, Amaravati, India. His research interests include machine learning, deep learning, natural language processing (NLP), and AI-accelerated software engineering and advanced approaches of multi-LLM chaining in applications like unit test generation. He is motivated to intervene and develop practical solutions with AI to address real-life challenges as a result, develop innovative solutions within these areas.

PYATLO BHUVANESWARI is currently pursuing the B.Tech. degree in CSE-AI with Amrita Vishwa Vidyapeetham, Amaravati Campus, India. Her research interests include machine learning, deep learning, and natural language processing (NLP). She looks forward to extending/engaging her research by being actively involved with projects and interdisciplinary research as a means to contribute to cutting-edge approaches in artificial intelligence.

MUKESH PRASAD received the M.S. degree from the School of Computer and System Sciences, Jawaharlal Nehru University, New Delhi, India, in 2009, and the Ph.D. degree from the Department of Computer Science, National Chiao Tung University, Taiwan, in 2015. He is currently a Senior Lecturer with the School of Computer Science, Faculty of Engineering and IT, UTS, and a Subject Matter Expert of artificial intelligence and machine learning with the Institute of Applied Technology (IAT), NSW, Australia. He has made substantial contributions to the fields of machine learning, artificial intelligence, data analytics, and natural language processing. He is also working in the evolving and increasingly important field of computer vision and the Internet of Things, which promise to pave the way for the evolution of new applications and services in the areas of healthcare, biomedical, agriculture, smart cities, education, marketing, and finance. His research has appeared in numerous prestigious journals, including IEEE/ACM Transactions and at conferences. He has published more than 200 research articles. He started his academic career as a lecturer with UTS, in 2017, and became a core member of Australian Artificial Intelligence Institute (AAII), the university's world-leading institute, which has a vision to develop theoretical foundations and advanced technologies for AI and to drive progress in related areas. His research is backed by industry experience, specifically in Taiwan, where he was the Principal Engineer at Taiwan Semiconductor Manufacturing Company (TSMC), from 2016 to 2017.

SUNIL KUMAR SINGH (Member, IEEE) received the B.Tech. and M.Tech. degrees in computer science and engineering and information technology from Kalyani Government Engineering College, Kalyani, India, in 2007 and 2010, respectively, and the Ph.D. degree in computer science and engineering from the National Institute of Technology Patna, Patna, India, in 2018. He is currently an Associate Professor with the Department of Computer Science and Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India. He has more than 35 publications in various national/international journals and conferences, such as IEEE, ACM, Springer, and Elsevier. His research interests include wireless sensor networks, the Internet of Things, and MANETs. He is a member of the program committee of various national/international conferences. He is a Guest Editor of *Computers and Electrical Engineering* journal (Elsevier). He has delivered expert talks and guest lectures at various prestigious institutes. He is a reviewer of several reputed journals indexed in SCI, SCIE, and Scopus.